SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303
Name: Duddela Harika **TOTAL: 95**
Reg No: 192324301

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

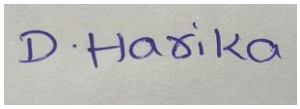
Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I **Duddela Harika / 192324301** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	24	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	24	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	19	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	14	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	19	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

TOTAL : 95

Answers:

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
    <input type="text" name="name">
<input type="email" name="email">
<button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure

The given HTML form is used to register students for an online symposium. However, the original

structure contains only a form and input fields. HTML5 semantic elements can be added to organize the webpage and improve its readability, accessibility, and maintainability.

The following semantic elements are appropriate:

1. <header>: Defines the introductory section of the webpage. It can contain the symposium title, college name, and a welcome message.
2. <nav>: Provides navigation links to different parts of the website, such as Home, About Symposium, Registration, and Contact.
3. <main>: Represents the main content of the webpage. It contains the primary information and registration form.
4. <section>: Groups related content under a common topic. In this portal, it can be used to organize the student registration form.
5. <footer>: Contains additional information such as copyright notices, contact details, and college information.
6. <form>: Groups the input controls and submits the student's registration details to the server.
7. <label>: Provides a descriptive name for each input field, helping users understand what information they need to enter.

These semantic elements create a meaningful document structure and make the portal easier to navigate for both users and assistive technologies.

2. Modify the structure to make it standards-compliant.

The original HTML structure can be improved by including the HTML5 document declaration, metadata, semantic elements, appropriate labels, and input validation.

HTML code

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Portal</title>
</head>
<body>

   <header>
      <h1>Online Symposium Registration</h1>
      <p>Register for our college symposium.</p>
   </header>

   <nav>
      <a href="#registration">Registration</a>
```

```
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="registration">
      <h2>Student Registration Form</h2>

      <form action="/register" method="POST">

        <label for="name">Full Name:</label>
        <input type="text" id="name"
            name="name" required>

        <br><br>

        <label for="email">Email Address:</label>
        <input type="email" id="email"
            name="email" required>

        <br><br>

        <button type="submit">Submit</button>

      </form>
    </section>
  </main>

  <footer id="contact">
    <p>Contact: symposium@example.com</p>
    <p>&copy; 2026 Online Symposium Portal</p>
  </footer>

</body>
</html>
```

Explanation

- <!DOCTYPE html> declares that the document uses HTML5.
- <html lang="en"> specifies the language of the webpage.
- <meta charset="UTF-8"> defines the character encoding.
- The viewport meta tag supports appropriate display on mobile devices.
- <title> specifies the title shown in the browser tab.
- <header> displays the symposium heading and introduction.
- <nav> provides links to the registration and contact sections.

- <main> contains the primary page content.
- <section> organizes the registration form.
- <label> elements improve accessibility by describing the input fields.
- required prevents submission when the fields are empty.
- type="email" enables basic browser validation of email formatting.
- method="POST" sends the form data in the HTTP request body.
- action="/register" specifies the server endpoint that processes the registration.

Note: The /register endpoint must be implemented on the server. HTML alone cannot store registration information in a database.

3. Interpret the HTTP request generated during form submission.

HTTP (Hypertext Transfer Protocol) is used for communication between a web browser and a web server. When a student fills in the registration form and clicks Submit, the browser sends an HTTP request to the server.

Because the form uses method="POST", the submitted information is included in the request body rather than appended to the URL.

For example, if the student enters a name and email address, the request may look like this:

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Harika&email=harika%40gmail.com
```

Explanation of the HTTP request

1. HTTP method: POST indicates that the browser is sending registration data to the server for processing.
2. Request URL: /register identifies the endpoint responsible for handling student registration.
3. HTTP version: HTTP/1.1 specifies the protocol version used in this example.
4. Host header: Host: example.com identifies the server receiving the request.
5. Content-Type header: application/x-www-form-urlencoded indicates that the form values are encoded as URL-style key-value pairs in the request body.
6. Request body: Contains the submitted name and email address. The @ symbol in the email is encoded as %40.
7. Server-side processing: The server receives the data, validates it, checks whether the registration

can be accepted, and may save it in a database.

The server should validate all submitted values because browser-side validation can be bypassed. HTTPS should also be used to protect the data during transmission.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle describes how a server processes a browser request and sends the result back to the user.

The cycle consists of the following steps:

1. Form submission: The student enters the required details and clicks the Submit button.
2. Request generation: The browser collects the form values and creates an HTTP POST request.
3. Request transmission: The browser sends the request to the server endpoint /register.
4. Server processing: The server receives the request, validates the submitted details, and checks the registration requirements.
5. Database operation: If the data is valid, the server may insert the registration record into a database.
6. Response generation: The server creates an HTTP response containing a status code, headers, and optionally a response body.
7. Response transmission: The server sends the response back to the browser.
8. Browser display: The browser displays the resulting page or message, informing the student whether registration succeeded or failed.

Example HTTP response

```
HTTP/1.1 201 Created
Content-Type: text/html

<h2>Registration Successful!</h2>
<p>Your symposium registration is complete.</p>
```

In this example, 201 Created indicates that a new registration resource has been created successfully. The response body contains a confirmation message.

Common HTTP status codes

- 200 OK: The request was processed successfully.
- 201 Created: A new registration record was created successfully.
- 400 Bad Request: The request contains invalid or unacceptable data.
- 404 Not Found: The requested endpoint does not exist.

- 500 Internal Server Error: An unexpected server-side error occurred.

The response status should reflect the actual result of the server operation. For example, a successful database insertion can return 201 Created, while invalid form data may return 400 Bad Request.

5. Suggest improvements for usability and accessibility.

The registration portal can be improved using the following techniques.

1. Use descriptive labels: Provide a label for every input field so users can understand what information is required. Labels also help screen readers identify form controls.
2. Implement input validation: Use HTML attributes such as required and type="email" to catch common input errors before submission. Additional validation must be performed on the server.
3. Provide clear feedback: Display success messages after registration and meaningful error messages when the submission fails. Preserve previously entered values when appropriate.
4. Design a responsive interface: Use CSS media queries, flexible layouts, and suitable input widths to make the form work on desktops, tablets, and mobile phones.
5. Support keyboard navigation: Ensure users can navigate the form using the Tab key and activate the Submit button using the keyboard.
6. Improve visual readability: Use readable font sizes, sufficient spacing, clear headings, and adequate contrast between text and background.
7. Use appropriate input types: Use type="email" for email addresses and suitable input types for any additional fields, such as telephone numbers or dates.
8. Protect registration data: Use HTTPS to encrypt data during transmission and validate all inputs on the server before storing them.
9. Prevent duplicate submissions: Disable repeated submission while processing when appropriate, and implement server-side safeguards against duplicate registrations.
10. Provide accessible error messages: Clearly identify fields that contain errors and explain how users can correct them.

2: Cross-Browser Compatibility Issue A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

1. Identify improper nesting and missing closing tags.

The given HTML snippet contains an incomplete webpage structure, which may lead to invalid markup and inconsistent rendering across browsers.

The issues are explained below:

1. Invalid heading syntax: The text # Welcome is not a valid HTML heading. In HTML, headings must be written using elements such as <h1>, <h2>, and <h3>.

2. Missing HTML structure: The document does not contain essential elements such as <html>, <head>, and <body>, which provide the basic structure of a webpage.

3. Missing title element: The <title> element is not included in the <head> section. It specifies the webpage title displayed in the browser tab.

4. Unstructured content: The text About us is not enclosed in an appropriate HTML element, such as a heading, paragraph, or navigation link.

5. Missing semantic elements: Elements such as <header>, <main>, <section>, and <footer> are absent. These elements help organize the webpage into meaningful areas.

6. Missing document declaration: The <!DOCTYPE html> declaration is required to ensure that modern browsers render the page using standards mode.

7. Missing character encoding: The <meta charset="UTF-8"> element should be included to support the correct display of text and special characters.

Therefore, the HTML code should be reorganized using valid elements, proper nesting, and a complete document structure.

2. Analyze how different browsers may interpret this structure.

Cross-browser compatibility means that a website should display and function consistently across different web browsers, such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

An incomplete HTML structure may cause the following problems:

1. Automatic error correction: Browsers often attempt to repair malformed HTML by inserting missing structural elements or adjusting the document tree. However, this behavior does not make the original HTML valid.

2. Differences in rendering: Browsers may apply different default styles to headings, paragraphs,

margins, and other elements, resulting in variations in the webpage's appearance.

3. Layout inconsistencies: Missing structural elements and incomplete markup can make it harder to control spacing, alignment, and content organization consistently.

4. CSS styling problems: CSS selectors may not behave as intended if the document structure differs from what the developer expects.

5. Accessibility issues: Screen readers and other assistive technologies rely on meaningful headings, landmarks, and properly structured content. Missing semantic elements can make navigation more difficult.

6. Responsive design problems: If the viewport declaration is missing, mobile browsers may use a wider layout viewport, causing the webpage to appear smaller or behave unexpectedly on mobile devices.

7. Maintenance difficulties: Invalid markup makes debugging more difficult and increases the likelihood of future layout errors.

To prevent these problems, developers should use valid HTML5, consistent CSS, and cross-browser testing techniques.

3. Identify missing semantic elements.

HTML5 semantic elements describe the meaning and purpose of different parts of a webpage. They improve document organization, accessibility, and maintainability.

The following semantic elements can be used:

1. <header>: Contains introductory content, such as the company name, logo, or main heading.

2. <nav>: Contains important navigation links that help users move between pages or sections.

3. <main>: Contains the primary content of the webpage. A document should generally have one visible main content area.

4. <section>: Groups related content under a common theme, usually with a heading.

5. <footer>: Contains information such as copyright notices, contact details, or related links.

6. <article>: Represents independent content that can stand on its own, such as a news article or blog post. It is optional for this simple company webpage.

Using these elements makes the document easier for developers to understand and helps assistive technologies identify important page regions.

4. Provide a corrected W3C-compliant HTML structure.

The following HTML5 code provides a complete and properly structured company webpage.

HTML code

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company Website</title>
</head>

<body>

   <header>
      <h1>Welcome</h1>
      <p>Welcome to our company website.</p>
   </header>

   <nav>
      <a href="#about">About Us</a>
   </nav>

   <main>
      <section id="about">
         <h2>About Us</h2>
         <p>
            Our company provides quality services
            and innovative solutions to customers.
         </p>
      </section>
   </main>

   <footer>
      <p>Copyright 2026 Company Website</p>
   </footer>

</body>

</html>
```

Explanation

- <!DOCTYPE html> tells the browser to use HTML standards mode.
- <html lang="en"> defines the document and its language.
- <head> contains metadata and the webpage title.
- <meta charset="UTF-8"> specifies the character encoding.
- The viewport meta tag helps the webpage display correctly on mobile devices.

- <header> contains the main heading and introductory content.
- <nav> provides a navigation link to the About Us section.
- <main> identifies the primary content of the page.
- <section> groups the About Us information.
- <footer> displays the copyright notice.

This structure follows standard HTML5 conventions and provides a strong foundation for consistent styling across browsers.

5. Suggest techniques to ensure cross-browser compatibility.

The following techniques help developers maintain consistent website behavior and appearance across browsers.

1. Use valid HTML5: Write properly nested elements, include required document structure, and close tags where required. Validate the markup using the W3C Nu HTML Checker.
2. Use CSS resets or normalization: Browsers may apply different default margins, padding, and styles. A CSS reset or normalization stylesheet helps reduce these differences.
3. Test across multiple browsers: Open the website in Chrome, Firefox, Edge, and Safari to identify layout, styling, and functionality issues.
4. Implement responsive design: Use flexible layouts, relative units, and CSS media queries to ensure that the website adapts to different screen sizes.
5. Check CSS and JavaScript compatibility: Verify that the properties and APIs used are supported by the target browsers. Use feature detection and suitable fallbacks when needed.
6. Use developer tools: Browser developer tools help identify HTML errors, CSS conflicts, JavaScript errors, and unexpected layout behavior.
7. Maintain consistent fonts and spacing: Specify suitable font families, font sizes, line heights, and spacing rather than relying entirely on browser defaults.
8. Validate HTML and CSS: Use standards-based validation tools to detect invalid markup and CSS syntax errors before deployment.
9. Test keyboard accessibility: Ensure that links and interactive elements can be accessed using the keyboard and that focus indicators remain visible.
10. Use automated testing: Browser automation tools can test important page elements and interactions across different browser environments.

3: Form Submission Failure A web application fails to send form data to the server.

The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

1. Identify issues in the form configuration.

The given HTML form contains several problems that prevent proper form submission and may affect security.

<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>

The issues are:

1. Incorrect button type: The button uses type="button", which does not automatically submit the form when clicked. It should use type="submit".
2. Inappropriate HTTP method: The form uses the GET method, which places submitted values in the URL as query parameters. This is unsuitable for login credentials.
3. Missing labels: The input fields do not have associated <label> elements, making the form less accessible to screen readers.
4. Missing validation: The input fields do not use the required attribute, so the browser can submit the form even when the fields are empty.
5. Missing autocomplete attributes: Appropriate attributes such as autocomplete="username" and autocomplete="current-password" can improve usability.
6. Security concerns: The form should use HTTPS to protect credentials during transmission. The server must also validate the submitted data and authenticate the user securely.

Therefore, the form requires corrections to its button type, HTTP method, validation, accessibility, and security.

2. Analyze why the HTTP request is not triggered.

The main reason the HTTP request is not triggered is the incorrect button type.

The given code contains:

<button type="button">Login</button>

The type="button" attribute creates a normal clickable button. It does not submit the form automatically unless JavaScript is used to perform that action.

The form submission process works as follows:

1. The user enters a username and password.
2. The user clicks the Login button.
3. Because the button has type="button", the browser does not automatically submit the form.
4. The browser does not send the expected HTTP request to the /submit endpoint.
5. The server does not receive the form data, and the login process does not begin.

To solve this problem, change the button to:

<button type="submit">Login</button>

When the user clicks this button, the browser validates the form controls and submits the form if validation succeeds. The browser then sends an HTTP request according to the form's action and method attributes.

Note: The GET method itself does not prevent a request from being sent. In this example, the primary issue is the button type.

3. Interpret the role of the GET method in this scenario.

The GET method is an HTTP method used mainly to retrieve information from a server. When a form uses GET, the browser appends the submitted form data to the URL as query parameters.

For example, if the user enters the username Harika and password 12345, the request URL could look like this:

GET /submit?username=Harika&password=12345 HTTP/1.1
Host: example.com

The roles and limitations of GET are:

1. Data transmission: Form values are appended to the URL as query parameters.
2. Data retrieval: GET is suitable for searches, filters, and requests that retrieve information.
3. URL visibility: Query parameters may appear in browser history, server logs, copied links, and other places.
4. Limited practicality for large data: URLs have practical length limits that vary across browsers and servers.
5. No confidentiality: GET does not encrypt data. HTTPS provides encryption, but sensitive values should still not be placed in URLs.
6. Unsuitable for login credentials: Sending passwords through GET can expose sensitive information.

For a login form, POST is more appropriate because the credentials are sent in the request body rather than appended to the URL.

However, POST alone does not encrypt the data. HTTPS is still necessary to protect credentials during transmission.

4. Propose a corrected implementation.

The following HTML5 code corrects the form submission issue and improves validation, accessibility, and usability.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Form</title>
</head>
<body>

   <header>
      <h1>User Login</h1>
   </header>

   <main>
      <section>
         <h2>Enter Your Login Details</h2>

         <form action="/submit" method="POST">

            <label for="username">Username:</label>
            <input
               type="text"
               id="username"
               name="username"
               autocomplete="username"
               required
            >

            <br><br>

            <label for="password">Password:</label>
            <input
               type="password"
               id="password"
               name="password"
               autocomplete="current-password"
               required
            >
```

```
            <br><br>

            <button type="submit">Login</button>

        </form>
      </section>
    </main>

    <footer>
      <p>Copyright 2026 User Login Portal</p>
    </footer>

</body>
</html>
```

Explanation of the corrected code:

- <!DOCTYPE html> declares the document as HTML5.
- <html lang="en"> identifies the language of the page.
- <header>, <main>, <section>, and <footer> provide a meaningful semantic structure.
- action="/submit" specifies the server endpoint that receives the form submission.
- method="POST" sends the form data in the request body.
- <label> elements provide accessible names for the input fields.
- name="username" and name="password" identify the submitted values.
- required prevents submission when either field is empty.
- autocomplete helps browsers identify the username and password fields.
- type="submit" triggers form submission.

When the form is submitted, the browser may generate a request similar to the following:

```
POST /submit HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=Harika&password=12345
```

The server receives the request, validates the input, checks the credentials securely, and returns an appropriate response. The /submit endpoint must be implemented on the server for the login process to work.

5. Suggest improvements for secure data transmission.

The login form can be improved through the following security measures:

1. Use HTTPS: HTTPS encrypts communication between the browser and server, helping prevent attackers from reading credentials during transmission.

2. Use the POST method: POST keeps credentials out of the URL query string. However, it does not independently provide encryption or guarantee security.

3. Perform server-side validation: The server must validate all incoming data because client-side validation can be bypassed.

4. Hash passwords securely: Passwords must not be stored in plain text. Use a password-hashing algorithm such as Argon2id, bcrypt, or scrypt with appropriate parameters.

5. Implement secure authentication: Verify the submitted credentials on the server and return a clear success or failure response without revealing unnecessary account information.

6. Protect user sessions: Use secure session cookies with appropriate HttpOnly, Secure, and SameSite attributes.

7. Prevent brute-force attacks: Apply rate limiting and other protections against repeated login attempts.

8. Avoid exposing sensitive information: Do not place passwords in URLs, application logs, error messages, or analytics records.

9. Use appropriate error messages: Display messages such as “Invalid username or password” without revealing which specific credential was incorrect.

10. Use secure password handling: Avoid unnecessary storage of credentials in browser storage such as local storage, and never log submitted passwords.

Result

The form submission failure is corrected by replacing type="button" with type="submit" and changing the HTTP method from GET to POST. Semantic HTML elements, accessible labels, and input validation improve usability. HTTPS, server-side validation, secure password hashing, and session protection help ensure secure login processing.